Объектно-ориентированное программирование

6 семестр
Лекция 3.

Содержание лекции:

- Инверсия управления (IoC)
- Паттерн Dependency Injection
- Сервисы

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Инверсия управления (IoC)

Inversion of Control (инверсия управления) — это некий абстрактный принцип, набор рекомендаций для написания слабо связанного кода. Суть которого в том, что каждый компонент системы должен быть как можно более изолированным от других, не полагаясь в своей работе на детали конкретной реализации других компонентов.

Преимущества:

- уменьшения зацепления (связанности);
- упрощающее расширение возможностей системы.

Программист может разместить свой код в определенных точках выполнения (используя callback или другие механизмы), затем запустить «главную функцию» фреймворка, которая обеспечит всё выполнение и вызовет код программиста тогда, когда это будет необходимо.
Как следствие, происходит утеря контроля над выполнением кода — это и называется инверсией управления.

Фреймворк управляет кодом программиста, а не программист управляет фреймворком.

Dependency Injection (внедрение зависимостей) — это одна из реализаций этого принципа (помимо этого есть еще Factory Method, Service Locator).

В центре этого механизма находится понятие **зависимость** - некоторая сущность, от которой зависит другая сущность.

Пример. Запись логов.

```
var logger = new Logger();
logger.Log("Hello");

class SimpleLogService
{
    public void Write(string message) =>
                                        Console.WriteLine(message);
}

class Logger
{
    SimpleLogService logService = new SimpleLogService();
    public void Log(string message) =>
                  logService.Write($"{DateTime.Now}  {message}");
}
```

Здесь имеется класс **Logger** - условный логгер, который логгирует некоторое сообщение с помощью метода **Log**(). При логгировании **Logger** добавляет к логгируемому сообщению дату.

Для логгирования **Logger** использует дополнительный класс **SimpleLogService**, который непосредственно управляет, как и куда будет логгироваться сообщение. В данном случае с помощью метода **Write** он просто выводит сообщение на консоль.

И при выполнении программы, как и ожидается, мы увидим на консоли сообщение, предваряемое датой:

```
15.11.2022 11:37:24  Hello
```

Данная программа прекрасно работает. Тем не менее в дальнейшем мы можем столкнуться с рядом проблем.

1. Прежде всего, класс **Logger** жестко привязан к классу **SimpleLogService**. И если мы захотим вместо **SimpleLogService** использовать другой тип логгера, например, логгировать в файл, а не на консоль, то нам придется менять класс **Logger**.
2. Если у нас в проекте много классов, которые используют **SimpleLogService** и с его помощью логгируют сообщения на консоль. Поменять во всех класс **Logger** на другой будет труднее.
3. Кроме того, класс **SimpleLogService** может иметь свои зависимости, которые тоже может потребоваться поменять.

Чтобы отвязать объект **Logger** от класса **SimpleLogService**, мы можем создать абстракцию, которая будет представлять сервис логгера, и передавать его извне в объект **Logger**:

```
interface ILogService
{
    void Write(string message);
}

// простой вывод на консоль
class SimpleLogService : ILogService
{
    public void Write(string message) =>
                                        Console.WriteLine(message);
}
```

```
// сервис, который выводит сообщение зеленым цветом
class GreenLogService : ILogService
{
    public void Write(string message)
    {
        var defaultColor = Console.ForegroundColor;
        Console.ForegroundColor = ConsoleColor.DarkGreen;
        Console.WriteLine(message);
        Console.ForegroundColor = defaultColor;
    }
}
```

```
class Logger
{
    ILogService logService;
    public Logger(ILogService logService) =>
                                    this.logService = logService;
    public void Log(string message) =>
                  logService.Write($"{DateTime.Now}  {message}");
}

var logger = new Logger(new SimpleLogService());
logger.Log("Hello METANIT.COM");

logger = new Logger(new GreenLogService());
logger.Log("Hello METANIT.COM");
```

Теперь класс **Logger** не зависит от конкретной реализации класса **SimpleLogService** - это может быть любая реализация интерфейса **ILogService**. Кроме того, создание сервиса логгера выносится во внешний код. Класс **Logger** больше ничего не знает о сервисе кроме того, что у него есть метод **Write**, который позволяет логгировать сообщение куда-то каким-то образом.

Для демонстрации я добавил второй класс сервиса логгера, который выводит сообщение на консоль зеленым цветом.

```
15.11.2022 13:40:53  Hello
15.11.2022 13:40:53  Hello
```

Тем не менее остается проблема управления подобными зависимостями, особенно если это касается больших приложений.

И если у нас десятки классов, которые через конструктор получают сервис логгирования, и мы хотим, чтобы они логгировали единообразно в едином стиле - на консоль, в файл, посылали по email и т.д., то опять же нам придется менять вызов конструктора этих классов.

Для упрощения управления зависимостями нередко используются специальные контейнеры - **IoC-контейнеры** (Inversion of Control). Такие контейнеры позволяют устанавливать зависимости между абстракциями и конкретными объектами и, как правило, управляют созданием этих объектов. Преимуществом .NET является то, что фреймворк имеет встроенный контейнер внедрения зависимостей, который представлен интерфейсом **IServiceProvider**. А сами зависимости еще называются сервисами, собственно поэтому контейнер можно назвать **провайдером сервисов**. Этот контейнер отвечает за сопоставление зависимостей с конкретными типами и за внедрение зависимостей в различные объекты.

Коллекция сервисов ServiceCollection

Все сервисы или зависимости хранятся в специальной коллекции сервисов, которая представляет тип **IServiceCollection**. .NET предоставляет встроенную реализацию этого интерфейса - класс **ServiceCollection**:

Для добавления сервисов в **ServiceCollection** применяется ряд методов. Например, добавим ранее определенный сервис **ILogService**:

```
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection()
    .AddTransient<ILogService, SimpleLogService>();
```

Здесь для добавления сервиса **ILogService** применяется метод **AddTransient<S, I>()**, который типизируется двумя типами.

Первый тип представляет сам сервис, а второй - его конкретную реализацию.

В данном случае мы говорим, что в качестве реализации сервиса **ILogService** будет выступать класс **SimpleLogService**.

Метод **AddTransient** возвращает измененный объект **IServiceCollection**.

Получение сервиса

Выше мы добавили сервис. Как теперь его получить и использовать в программе?

Для этого нам нужен провайдер сервисов **IServiceProvider**. Для его получения у коллекции сервисов вызывается метод **BuildServiceProvider()**, который возвращает встроенную реализацию провайдера - объект **ServiceProvider**:

```
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection()
    .AddTransient<ILogService, SimpleLogService>();

using var serviceProvider = services.BuildServiceProvider();

// получаем сервис ILogService
ILogService? logService =
                        serviceProvider.GetService<ILogService>();

// используем сервис
logService?.Write("Hello");
```

Таким образом, у нас определена единая точка, где мы определяем конкретную реализацию сервиса - метод **AddTransient**.

```
var services = new ServiceCollection()
    .AddTransient<ILogService, SimpleLogService>();
```

И в программе в любой ее точке мы можем вызвать метод **GetService** и получить реализацию сервиса.

```
ILogService? logService =
                        serviceProvider.GetService<ILogService>();
```

Получение сервиса в конструкторе

Более предпочтительным способом передачи зависимостей в классы представляет использование конструктора. Например, определим класс Logger:

```
class Logger
{
    ILogService? logService;
    public Logger(ILogService? logService) =>
                                    this.logService = logService;
    public void Log(string message) =>
                  logService?.Write($"{DateTime.Now}  {message}");
}
```

Класс **Logger** получает через конструктор сервис **ILogService** и для логгирования сообщения в методе **Log** вызывает его метод **Write**. Затем в программе мы можем явным образом создать объект этого класса, передав в конструктор нужную реализацию **ILogService**:

```
var services = new ServiceCollection()
    .AddTransient<ILogService, SimpleLogService>();

using var serviceProvider = services.BuildServiceProvider();

ILogService? logService =
                        serviceProvider.GetService<ILogService>();

Logger logger = new Logger(logService); // создаем объект Logger

logger.Log("Hello");
```

Но также мы можем для создания объекта **Logger** использовать тот же механизм внедрения зависимостей:

```
interface ILogService
{
    void Write(string message);
}

class SimpleLogService : ILogService
{
    public void Write(string message) =>
                                        Console.WriteLine(message);
}
```

```
class Logger
{
    ILogService? logService;
    public Logger(ILogService? logService) =>
                                    this.logService = logService;
    public void Log(string message) =>
                logService?.Write($"{DateTime.Now}  {message}");
}

var services = new ServiceCollection()
    .AddTransient<ILogService, SimpleLogService>()
    .AddTransient<Logger>();

using var serviceProvider = services.BuildServiceProvider();

// получаем объект Logger
Logger? logger = serviceProvider.GetService<Logger>();
logger?.Log("Hello");
```

Обратите внимание, что здесь нигде явным образом мы не определяем объект **ILogService**, который передается в конструктор объекта **Logger**, это делает за нас система внедрения зависимостей. Она видит, что для сервиса **ILogService** зарегистрирована реализация **SimpleLogService**, поэтому при создании объекта **Logger** неявно создает объект **SimpleLogService** и передает его в конструктор **Logger**.

```
Logger? logger = serviceProvider.GetService<Logger>();
```

Жизненный цикл зависимостей

Встроенный механизм внедрения зависимостей .NET позволяет управлять жизненным циклом сервисов. С точки зрения жизненного цикла сервисы могут представлять один из следующих типов:

- **Transient**: при каждом обращении к сервису создается новый объект сервиса.
- **Scoped**: создается отдельный контекст или scope, и в рамках этого контекста при всех обращениях у сервису будет использоваться один и тот же объект сервиса.
- **Singleton**: объект сервиса создается при первом обращении к нему, все последующих обращениях используется один и тот же ранее созданный объект сервиса.

Для создания каждого типа сервиса предназначен соответствующий метод **AddTransient()**, **AddScoped()** и **AddSingleton()**.

Пример:

```
public interface ICounter
{
    int Value { get; }
}

public class RandomCounter : ICounter
{
    static Random rnd = new Random();
    private int _value;
    public RandomCounter() => _value = rnd.Next(0, 1000000);
    public int Value => _value;
}
```

Интерфейс **ICounter** представляет сервис, который в свойстве **Value** хранит некоторое число. Реализацию этого интерфейса - класс **RandomCounter** использует класс **Random** для генерации случайного числа в диапазоне от **0** до **1000000** и возвращает это число из свойства **Value**.

Теперь на примере рассмотрим уравление жизненным циклом сервисов.

Для большей показательности я вынес логику по получению сервиса в отдельный метод **PrintCounters**. Причем в каждом методе два раза получаем сервис **ICounter** и выводим на консоль значение его свойства **Value**.

```
void PrintCounters()
{
    var counter1 = serviceProvider.GetService<ICounter>();
    var counter2 = serviceProvider.GetService<ICounter>();

    Console.WriteLine(
     $"Counter1: {counter1?.Value}; Counter2: {counter2?.Value}");
}
```

AddTransient

Метод **AddTransient()** создает transient-объекты. Такие объекты создаются при каждом обращении к ним.

```
var services = new ServiceCollection()
    .AddTransient<ICounter, RandomCounter>();

using ServiceProvider serviceProvider =
                                services.BuildServiceProvider();

PrintCounters();
PrintCounters();
```

```
Counter1: 939614; Counter2: 862579
Counter1: 741773; Counter2: 730321
```

Как видно, в программе идет 4 обращения к провайдеру сервисов для получения сервиса **ICounter**. И при каждом обращении создается свой объект **RandomCounter**. Соответственно во всех четырех случаях случайные числа будут разные.

```
Counter1: 939614; Counter2: 862579
Counter1: 741773; Counter2: 730321
```

AddSingleton

Для сервисов, которые добавляются с помощью метода **AddSingleton()** создается один объект для всех последующих обращений.

```
using Microsoft.Extensions.DependencyInjection;

IServiceCollection services = new ServiceCollection()
    .AddSingleton<ICounter, RandomCounter>();

using ServiceProvider serviceProvider =
services.BuildServiceProvider();

PrintCounters();
PrintCounters();
```

```
Counter1: 650427; Counter2: 650427
Counter1: 650427; Counter2: 650427
```

Здесь та же самая логика, только теперь сервис **ICounter** определен как синглтон, соответственно при каждом обращении к провайдеру сервисов получим один и тот же объект **ICounter**:

```
Counter1: 650427; Counter2: 650427
Counter1: 650427; Counter2: 650427
```

AddScoped

Для сервисов, которые добавляются методом **AddScoped**, будет создаваться один экземпляр объекта для одного контекста или **scope**. Здесь под контекстом понимает область видимости объекта **IServiceScope**. Для получения этого объекта у **ServiceProvider** вызывается метод **CreateScope**():

```
using ServiceProvider serviceProvider =
                                services.BuildServiceProvider();

using (IServiceScope scope = serviceProvider.CreateScope())
{
    // контекст IServiceScope
}
```

```
IServiceCollection services = new ServiceCollection()
    .AddScoped<ICounter, RandomCounter>();

using ServiceProvider serviceProvider =
                                services.BuildServiceProvider();

PrintCounters();
PrintCounters();

void PrintCounters()
{
    using IServiceScope scope = serviceProvider.CreateScope();

    var counter1 = scope.ServiceProvider.GetService<ICounter>();
    var counter2 = scope.ServiceProvider.GetService<ICounter>();
    Console.WriteLine(
     $"Counter1: {counter1?.Value}; Counter2: {counter2?.Value}");
}
```

Здесь контекст одного объекта **IServiceScope** ограничивается методом **PrintCounters()**. Для получения сервиса в пределах этого контекста сначла получаем у **scope** провайдер сервиса и через него получаем сервис **ICounter**:

```
using IServiceScope scope = serviceProvider.CreateScope();

var counter1 = scope.ServiceProvider.GetService<ICounter>();
```

Поскольку здесь метод **PrintCounters** вызывается два раза, соответственно будут создаваться два разных контекста. И в рамках каждого контекста мы будем получать свой объект **ICounter**:

```
Counter1: 620832; Counter2: 620832
Counter1: 816555; Counter2: 816555
```